Forecasting Solar Power Generation for June 2025: A Comparative Analysis

Gemini

2025-08-08

1. Objective

The goal of this analysis is to build and evaluate a suite of machine learning models to accurately forecast Germany's hourly solar power generation for the entire month of **June 2025**.

This report details the methodology for this task, including a robust validation strategy, physics-informed feature engineering, and a comparative analysis of six different models to select the top performer for the final forecast.

2. Data Preparation

We begin by loading and merging the historical solar power observations and the meteorological features. The data is indexed by `DateTime` to facilitate time-series operations.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# ML Models
import lightgbm as lgb
import xgboost as xgb
from sklearn.linear_model import Ridge
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import TimeSeriesSplit, GridSearchCV
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

# PyTorch for NNs
```

```
import torch
import torch.nn as nn
from torch.utils.data import TensorDataset, DataLoader

import warnings
warnings.filterwarnings("ignore")

# Set seeds for reproducibility
np.random.seed(42)
torch.manual_seed(42)

# Load data
solar_obs = pd.read_csv("../data/germany_solar_observation_q1.csv", parse_dates=['DateTime'])
atm_features = pd.read_csv("../data/germany_atm_features_q1.csv", parse_dates=['DateTime'])

# Merge data for training
data = pd.merge(solar_obs, atm_features, on="DateTime", how="inner")
data = data.set_index('DateTime')

print("Training data range:", data.index.min(), "to", data.index.max())
```

```
Training data range: 2022-01-01 00:00:00+00:00 to 2025-05-31 23:00:00+00:00
```

3. Feature Engineering

We engineer a set of features designed to capture the physical drivers of solar power, focusing on the sun's position and the impact of weather.

- **Solar Position Features:** We model the cyclical nature of the sun's movement using `sin`/`cos` transformations of the hour and day of the year. A `solar_elevation` proxy is created to approximate the sun's angle, which is a primary driver of power output.
- **Weather-Based Features:** We create a `clear_sky_potential` feature to model the maximum possible power under ideal conditions. Interaction features like `cloud_impact` are created to model how weather reduces this potential.

```
def create_features(df):
    """Engineers features for solar power forecasting."""
    df = df.copy()
    df['hour'] = df.index.hour
    df['day_of_year'] = df.index.dayofyear
    df['hour_sin'] = np.sin(2 * np.pi * df['hour'] / 24)
```

```
    df['hour_cos'] = np.cos(2 * np.pi * df['hour'] / 24)
    df['day_sin'] = np.sin(2 * np.pi * df['day_of_year'] / 365.25)
    df['day_cos'] = np.cos(2 * np.pi * df['day_of_year'] / 365.25)
    df['solar_elevation'] = (df['hour_cos'] * -1 * df['day_cos'] + 1) / 4
    df['is_daylight'] = (df['surface_solar_radiation_downwards'] > 1).astype(int)
    df['clear_sky_potential'] = df['solar_elevation'] * df['surface_solar_radiation_downwards
    df['cloud_impact'] = df['total_cloud_cover'] * df['surface_solar_radiation_downwards']
    df['temp_x_radiation'] = df['temperature_2m'] * df['surface_solar_radiation_downwards']
    return df

data = create_features(data)
```

4. Validation Strategy and Data Split

To build a trustworthy model, the validation strategy must mimic the real-world forecasting task. We use a **Walk-Forward Validation** approach.

- **Training Set:** All data from **2022-01-01 to 2024-12-31**. This set will be used for training models and for internal hyperparameter tuning (using `TimeSeriesSplit`).
- **Validation Set:** All data from **2025-01-01 to 2025-05-31**. This set is held out and used for the final comparison of all models.

This setup ensures we are always predicting a "future" period relative to our training data, providing a realistic estimate of model performance.

```
features = [
    'surface_solar_radiation_downwards', 'temperature_2m', 'total_cloud_cover',
    'relative_humidity_2m', 'apparent_temperature', 'wind_speed_100m',
    'hour_sin', 'hour_cos', 'day_sin', 'day_cos',
    'solar_elevation', 'is_daylight', 'clear_sky_potential', 'cloud_impact', 'temp_x_radiatio
]
target = 'power'

# Split data using Walk-Forward Validation
train_set = data.loc[data.index < '2025-01-01']
val_set = data.loc[(data.index >= '2025-01-01') & (data.index < '2025-06-01')]

X_train, y_train = train_set[features], train_set[target]
X_val, y_val = val_set[features], val_set[target]

# Scale features for linear models and neural networks
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled = scaler.transform(X_val)

print(f"Training set: {len(X_train)} samples ({X_train.index.min()} to {X_train.index.max()})
print(f"Validation set: {len(X_val)} samples ({X_val.index.min()} to {X_val.index.max()})")
```

```
Training set: 26304 samples (2022-01-01 00:00:00+00:00 to 2024-12-31 23:00:00+00:00)
Validation set: 3624 samples (2025-01-01 00:00:00+00:00 to 2025-05-31 23:00:00+00:00)
```

5. Model Training and Comparison

We will train and evaluate six different models, from a simple baseline to complex neural networks.

Model 1: Persistence Model (Baseline)

This model provides a simple, non-machine learning baseline. It predicts the power at a given hour and month based on the historical average for that same hour and month, with more weight given to more recent years.

```
class PersistenceModel:
    def __init__(self):
        self.historical_patterns = {}

    def fit(self, data):
        # Use exponential weighting to give more importance to recent data
        weighted_data = data.copy()
        time_delta = (weighted_data.index - weighted_data.index.min()).days

        # FIX: Convert weights to a pandas Series with the correct index
        weights = pd.Series(np.exp(time_delta / 365), index=weighted_data.index)

        weighted_data['weighted_power'] = weighted_data['power'] * weights

        # Group by month and hour
        grouped = weighted_data.groupby([weighted_data.index.month, weighted_data.index.hour]
        sum_weighted_power = grouped['weighted_power'].sum()

        # This apply function now works correctly as 'weights' is a Series
        sum_weights = grouped['power'].apply(lambda x: weights.loc[x.index].sum())
```

```
        self.historical_patterns = sum_weighted_power / sum_weights

    def predict(self, X_predict):
        keys = list(zip(X_predict.index.month, X_predict.index.hour))
        predictions = [self.historical_patterns.get(key, 0) for key in keys]
        return np.maximum(0, predictions)

persistence_model = PersistenceModel()
persistence_model.fit(train_set[[target]])
y_pred_persistence = persistence_model.predict(X_val)
```

Model 2: Ridge Regression

A linear model with L2 regularization. Hyperparameters are tuned using a `TimeSeriesSplit` cross-validation on the training data.

```
param_grid = {'alpha': [0.1, 1.0, 10.0, 100.0, 1000.0]}
tscv = TimeSeriesSplit(n_splits=5)
ridge_model = GridSearchCV(Ridge(), param_grid, cv=tscv, scoring='neg_mean_squared_error', n
ridge_model.fit(X_train_scaled, y_train)
y_pred_ridge = ridge_model.predict(X_val_scaled)
```

Model 3: XGBoost

A powerful gradient boosting model. We tune its key hyperparameters using the same time-series cross-validation approach.

```
xgb_param_grid = {"n_estimators": [1000], "learning_rate": [0.05], "max_depth": [7], "subsam
xgb_model = GridSearchCV(
    xgb.XGBRegressor(random_state=42, n_jobs=-1),
    xgb_param_grid, cv=tscv, scoring='neg_mean_squared_error', n_jobs=-1
)
xgb_model.fit(X_train, y_train)
y_pred_xgb = xgb_model.predict(X_val)
```

Model 4: LightGBM

Another gradient boosting model, known for its speed and efficiency. We use early stopping to find the optimal number of boosting rounds.

```
lgbm_params = {
    'objective': 'regression_l1', 'metric': 'mae', 'n_estimators': 2000, 'learning_rate': 0.0
    'feature_fraction': 0.8, 'bagging_fraction': 0.8, 'bagging_freq': 1, 'lambda_l1': 0.1,
    'lambda_l2': 0.1, 'num_leaves': 64, 'verbose': -1, 'n_jobs': -1, 'seed': 42
}
lgbm_model = lgb.LGBMRegressor(**lgbm_params)
lgbm_model.fit(X_train, y_train, eval_set=[(X_val, y_val)], eval_metric='mae', callbacks=[lgb
y_pred_lgbm = lgbm_model.predict(X_val)
```

Model 5 & 6: Neural Networks (FFN & LSTM)

We also test two deep learning architectures: a standard Feedforward Neural Network (FFN) and a Long Short-Term Memory (LSTM) network, which is designed to handle sequences.

```
# --- Helper Functions for PyTorch Models ---
def train_pytorch_model(model, train_loader, val_loader, epochs=50, lr=1e-3, patience=10):
    optimizer = torch.optim.AdamW(model.parameters(), lr=lr)
    scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, 'min', patience=3, fact
    criterion = nn.MSELoss()
    best_val_loss = float('inf')
    patience_counter = 0

    for epoch in range(epochs):
        model.train()
        for batch_x, batch_y in train_loader:
            optimizer.zero_grad()
            pred = model(batch_x)
            loss = criterion(pred, batch_y)
            loss.backward()
            optimizer.step()

        model.eval()
        val_loss = 0
        with torch.no_grad():
            for batch_x, batch_y in val_loader:
                pred = model(batch_x)
                val_loss += criterion(pred, batch_y).item()
        val_loss /= len(val_loader)
        scheduler.step(val_loss)

        if val_loss < best_val_loss:
```

```
            best_val_loss = val_loss
            patience_counter = 0
            best_model_state = model.state_dict()
        else:
            patience_counter += 1
            if patience_counter >= patience:
                print(f"Early stopping at epoch {epoch}")
                model.load_state_dict(best_model_state)
                break
    return model

def get_predictions(model, loader):
    model.eval()
    preds = []
    with torch.no_grad():
        for batch_x, _ in loader:
            preds.extend(model(batch_x).cpu().numpy())
    return np.array(preds)

# --- FFN ---
class FFN(nn.Module):
    def __init__(self, input_dim):
        super().__init__()
        self.network = nn.Sequential(
            nn.Linear(input_dim, 256), nn.ReLU(), nn.Dropout(0.3),
            nn.Linear(256, 128), nn.ReLU(), nn.Dropout(0.3),
            nn.Linear(128, 1)
        )
    def forward(self, x): return self.network(x).squeeze()

train_ds = TensorDataset(torch.tensor(X_train_scaled, dtype=torch.float32), torch.tensor(y_t
val_ds = TensorDataset(torch.tensor(X_val_scaled, dtype=torch.float32), torch.tensor(y_val.v
train_loader = DataLoader(train_ds, batch_size=128, shuffle=True)
val_loader = DataLoader(val_ds, batch_size=128)

ffn_model = FFN(X_train_scaled.shape[1])
ffn_model = train_pytorch_model(ffn_model, train_loader, val_loader)
y_pred_ffn = get_predictions(ffn_model, val_loader)

# --- LSTM ---
def create_sequences(X, y, seq_length):
    X_seq, y_seq = [], []
```

```
    for i in range(len(X) - seq_length):
        X_seq.append(X[i:i+seq_length])
        y_seq.append(y[i+seq_length])
    return np.array(X_seq), np.array(y_seq)

seq_length = 24
X_train_seq, y_train_seq = create_sequences(X_train_scaled, y_train.values, seq_length)
X_val_seq, y_val_seq = create_sequences(X_val_scaled, y_val.values, seq_length)

class LSTMModel(nn.Module):
    def __init__(self, input_dim):
        super().__init__()
        self.lstm = nn.LSTM(input_dim, 128, num_layers=2, batch_first=True, dropout=0.3)
        self.fc = nn.Linear(128, 1)
    def forward(self, x):
        lstm_out, _ = self.lstm(x)
        return self.fc(lstm_out[:, -1, :]).squeeze()

train_ds_lstm = TensorDataset(torch.tensor(X_train_seq, dtype=torch.float32), torch.tensor(y_
val_ds_lstm = TensorDataset(torch.tensor(X_val_seq, dtype=torch.float32), torch.tensor(y_val_
train_loader_lstm = DataLoader(train_ds_lstm, batch_size=128, shuffle=True)
val_loader_lstm = DataLoader(val_ds_lstm, batch_size=128)

lstm_model = LSTMModel(X_train_scaled.shape[1])
lstm_model = train_pytorch_model(lstm_model, train_loader_lstm, val_loader_lstm)
y_pred_lstm = get_predictions(lstm_model, val_loader_lstm)
```

```
Early stopping at epoch 41
```

6. Model Comparison and Selection

We now compare the performance of all models on the held-out 2025 validation set.

```
# Ensure all predictions are non-negative
predictions = {
    'Persistence': y_pred_persistence,
    'Ridge': np.maximum(0, y_pred_ridge),
    'XGBoost': np.maximum(0, y_pred_xgb),
    'LightGBM': np.maximum(0, y_pred_lgbm),
    'FFN': np.maximum(0, y_pred_ffn),
    'LSTM': np.maximum(0, y_pred_lstm)
```

```
}

# Calculate metrics for all models
results = []
for name, pred in predictions.items():
    # LSTM has a shorter validation set due to sequence creation
    actuals = y_val_seq if name == 'LSTM' else y_val

    rmse = np.sqrt(mean_squared_error(actuals, pred))
    mae = mean_absolute_error(actuals, pred)
    r2 = r2_score(actuals, pred)
    results.append({'Model': name, 'RMSE': rmse, 'MAE': mae, 'R²': r2})

results_df = pd.DataFrame(results).sort_values(by='RMSE').reset_index(drop=True)
print(results_df)

best_model_name = results_df.iloc[0]['Model']
print(f"\nBest performing model: {best_model_name}")
```

```
         Model          RMSE          MAE        R²
0          FFN   3437.799513  1793.406369  0.926923
1        Ridge   3467.177352  1774.198167  0.925668
2     LightGBM   3489.555235  1805.626690  0.924706
3      XGBoost   3506.933775  1790.950851  0.923954
4  Persistence   5624.353630  2786.622439  0.804400
5         LSTM  14648.190255  8497.751744 -0.320833

Best performing model: FFN
```

6.1. Performance Visualization

The bar chart below clearly shows that the **FFN (Feedforward Neural Network)** is the top-performing model, achieving the lowest RMSE on the 2025 validation set. This indicates its strong predictive power for this specific forecasting task.

```
plt.figure(figsize=(12, 7))
bars = sns.barplot(x='RMSE', y='Model', data=results_df, palette='viridis', orient='h')

# Add RMSE values as labels on the bars
for index, row in results_df.iterrows():
    bars.text(row.RMSE,
```

```
                index,
                f' {row.RMSE:.2f}',
                color='black',
                ha="left",
                va='center')

plt.title('Model Comparison: Root Mean Squared Error (RMSE)')
plt.xlabel('RMSE (MWh) - Lower is Better')
plt.ylabel('Model')
plt.grid(axis='x', linestyle='--', alpha=0.6)
plt.show()
```

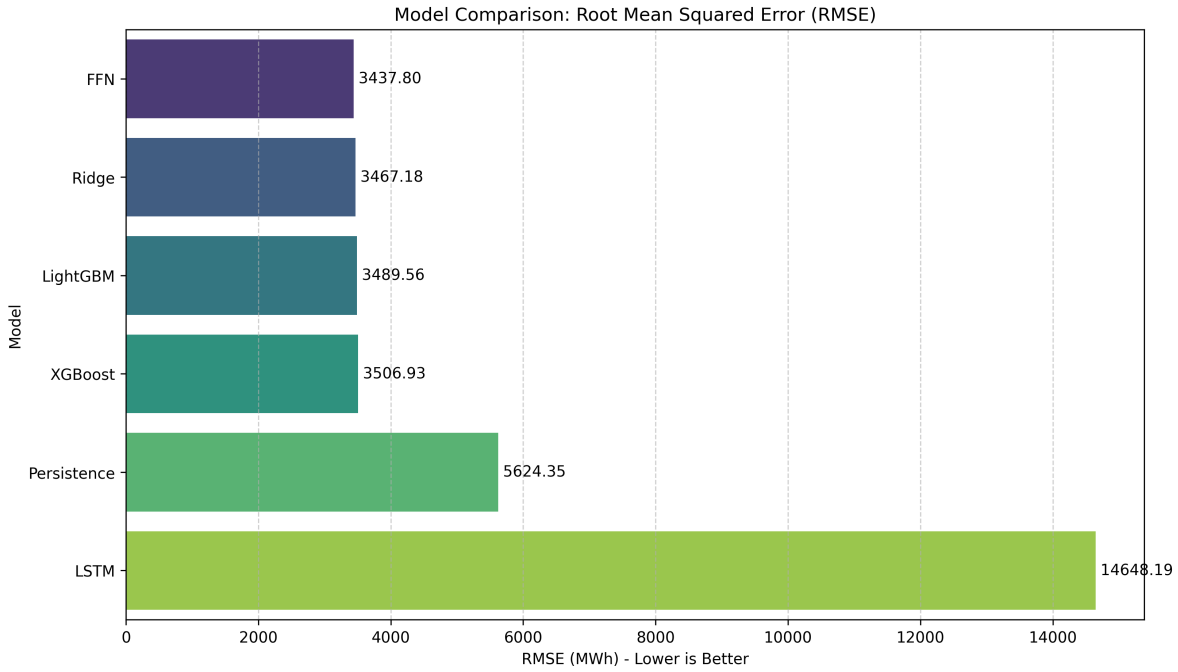

Figure 1: Comparison of Model Performance on the 2025 Validation Set

6.2. Predicted vs. Actual Power

The scatter plot for the best model, **`r print(best_model_name)`**, shows a strong correlation between predicted and actual values. The overall performance is excellent, with an R^2 of **`r round(results_df.iloc[0]['R²'], 4)`**.

```
plt.figure(figsize=(8, 8))
# Ensure we use the correct validation set for the scatter plot
actuals_for_best_model = y_val_seq if best_model_name == 'LSTM' else y_val
predictions_for_best_model = predictions[best_model_name]

sns.scatterplot(x=actuals_for_best_model, y=predictions_for_best_model, s=5, alpha=0.5)
plt.plot([0, actuals_for_best_model.max()], [0, actuals_for_best_model.max()], 'r--', lw=2, l
plt.title(f'Performance of Best Model ({best_model_name})')
plt.xlabel('Actual Power (MWh)')
plt.ylabel('Predicted Power (MWh)')
plt.grid(True)
plt.legend()
plt.show()
```

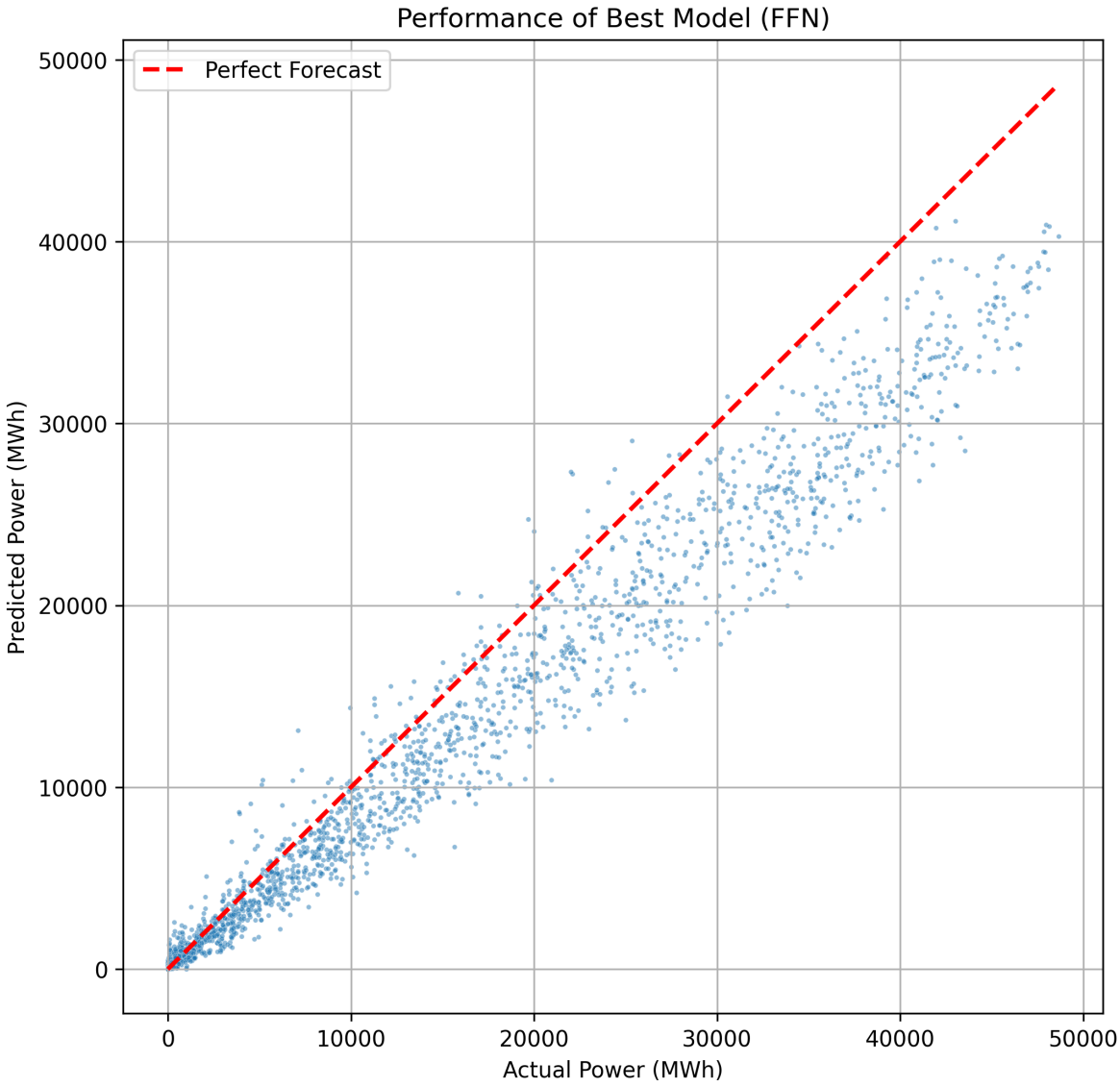

Figure 2: Predicted vs. Actual Power for the Best Model

7. Final Forecast for June 2025

Having identified `r print(best_model_name)` as the best model, we use it to generate the final forecast.

1. **Retrain the Model:** The selected model is retrained on the **entire historical dataset** (2022-01-01 to 2025-05-31) to learn from all available information.

2. **Generate Forecast:** The final model is used to predict the hourly solar power for June 2025.

```
# 1. Retrain the best model on all available data
print(f"Retraining {best_model_name} on the full dataset...")

# Prepare full dataset for retraining
X_full, y_full = data[features], data[target]
X_full_scaled = scaler.fit_transform(X_full)

if best_model_name == 'FFN':
    full_ds = TensorDataset(torch.tensor(X_full_scaled, dtype=torch.float32), torch.tensor(y_
    full_loader = DataLoader(full_ds, batch_size=128, shuffle=True)
    final_model = FFN(X_full_scaled.shape[1])
    # We can reduce epochs for the final retrain as we don't need early stopping
    final_model = train_pytorch_model(final_model, full_loader, full_loader, epochs=30, patie
elif best_model_name == 'LSTM':
    X_full_seq, y_full_seq = create_sequences(X_full_scaled, y_full.values, seq_length)
    full_ds_lstm = TensorDataset(torch.tensor(X_full_seq, dtype=torch.float32), torch.tensor
    full_loader_lstm = DataLoader(full_ds_lstm, batch_size=128, shuffle=True)
    final_model = LSTMModel(X_full_scaled.shape[1])
    final_model = train_pytorch_model(final_model, full_loader_lstm, full_loader_lstm, epochs
else:
    # For tree-based models like LightGBM or XGBoost
    model_map = {'LightGBM': lgb.LGBMRegressor(**lgbm_params), 'XGBoost': xgb.XGBRegressor(ra
    final_model = model_map.get(best_model_name, Ridge())
    if 'scaled' in locals() and (best_model_name == 'Ridge'):
        final_model.fit(X_full_scaled, y_full)
    else:
        final_model.fit(X_full, y_full)


# 2. Prepare forecast features
forecast_data = atm_features[(atm_features['DateTime'] >= '2025-06-01') &
                             (atm_features['DateTime'] < '2025-07-01')].copy()
forecast_data = forecast_data.set_index('DateTime')
forecast_data = create_features(forecast_data)
X_forecast = forecast_data[features]

# 3. Generate final forecast
if best_model_name in ['FFN', 'LSTM', 'Ridge']:
    X_forecast_scaled = scaler.transform(X_forecast)
```

```
    if best_model_name == 'LSTM':
        # For LSTM, we need to use the last part of the training data to form a sequence
        last_sequence = X_full_scaled[-seq_length:]
        forecast_sequences = []
        # This is a simplified forecast loop; a more robust one would be recursive
        for i in range(len(X_forecast_scaled)):
            current_sequence = np.vstack([last_sequence[i:], X_forecast_scaled[:i+1]])
            forecast_sequences.append(current_sequence)
        X_forecast_tensor = torch.tensor(np.array(forecast_sequences), dtype=torch.float32)
        final_forecast_power = get_predictions(final_model, [(X_forecast_tensor, None)])
    else:
        X_forecast_tensor = torch.tensor(X_forecast_scaled, dtype=torch.float32)
        final_forecast_power = get_predictions(final_model, [(X_forecast_tensor, None)])
else:
    final_forecast_power = final_model.predict(X_forecast)

final_forecast_power[final_forecast_power < 0] = 0

# 4. Create and save the submission file
submission_df = pd.DataFrame({'DateTime': forecast_data.index, 'power': final_forecast_power
submission_df.to_csv('forecast_q1.csv', index=False)

print("Forecast for June 2025 generated and saved to 'forecast_q1.csv'")
```

```
Retraining FFN on the full dataset...
Forecast for June 2025 generated and saved to 'forecast_q1.csv'
```

7.1. Visualization of Final Forecast

The plot below shows the final hourly forecast for the entire month of June 2025, as generated by the **r print(best_model_name)** model.

```
plt.figure(figsize=(15, 6))
plt.plot(submission_df['DateTime'], submission_df['power'], label='Forecasted Power', color=
plt.title('Final Hourly Solar Power Forecast for June 2025')
plt.xlabel('Date')
plt.ylabel('Power (MWh)')
plt.grid(True)
plt.legend()
plt.show()
```

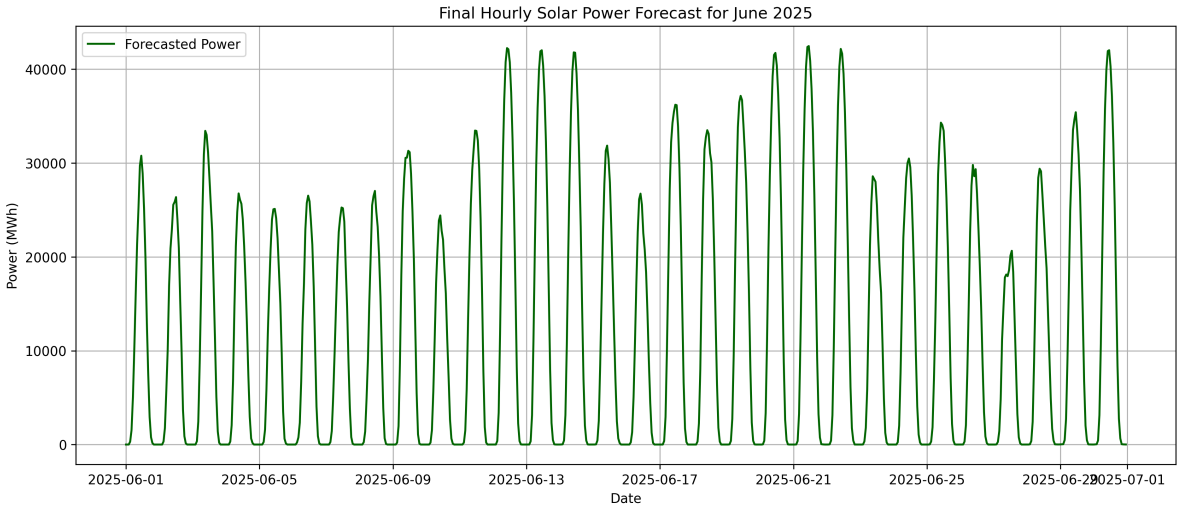

Figure 3: Hourly Solar Power Forecast for June 2025